

Semana 1 — Benchmark 3D y Comprensión del Modelo

Laboratorio Estructural Digital del Edificio de Ingeniería
Universidad de los Andes

Grupo 7

Integrantes: Pedro Castillo, Monserrat Cubillos, Eduardo Vergara

27 de agosto de 2026

1. Modelo Entregado

1.1. Geometría

El modelo corresponde al **Edificio de Ingeniería de la Universidad de los Andes**, idealizado a partir de planos reales en AutoCAD (.dwg). La geometría fue extraída de los archivos DXF usando la librería `ezdxf` en Python.

Parámetro	Valor
Dimensiones en planta	45,0 m × 25,8 m
Altura total	28,5 m
Niveles	9 (fundación + 8 pisos)
Ejes verticales (X)	8 ejes
Ejes horizontales (Y)	6 ejes
Nodos por piso	48
Total de nodos	432

Cuadro 1: Parámetros globales del modelo.

Ejes en dirección X (m): 8,02, 11,32, 14,72, 18,02, 28,02, 38,02, 48,02, 53,02

Ejes en dirección Y (m): 46,92, 50,26, 55,20, 60,20, 65,22, 72,75

Niveles (Z , m): 0,0 (fundación), 4,0, 7,5, 11,0, 14,5, 18,0, 21,5, 25,0, 28,5

1.2. Elementos

Tipo	Cantidad	Sección (cm)	Material
Columnas	384	50 × 50	Hormigón $f'_c = 28$ MPa
Vigas dirección X	336	30 × 60	Hormigón $f'_c = 28$ MPa
Vigas dirección Y	320	30 × 80	Hormigón $f'_c = 28$ MPa
Total	1 040	—	—

Cuadro 2: Resumen de elementos estructurales.

Todos los elementos son `elasticBeamColumn` con 6 GDL por nodo.

1.3. Apoyos

- **48 apoyos fijos** en el nivel de fundación ($Z = 0,0$ m).
- Cada apoyo restringe los 6 GDL: traslaciones (U_x, U_y, U_z) y rotaciones ($\theta_x, \theta_y, \theta_z$).
- En OpenSees: `fix(nodTag, 1, 1, 1, 1, 1, 1)`.

1.4. Cargas

Caso	Descripción	Intensidad
G	Muerta propia + losa + terminaciones	$7,75 \text{ kN/m}^2$
Q	Viva	$2,0 \text{ kN/m}^2$
E_X	Sismo lateral en X	Triángulo invertido, $10 \times N$ (kN)
E_Y	Sismo lateral en Y	Triángulo invertido, $10 \times N$ (kN)

Cuadro 3: Casos de carga.

Carga gravitacional:

- Peso propio de losa: $25 \text{ kN/m}^3 \times 0,25 \text{ m} = 6,25 \text{ kN/m}^2$
- Terminaciones: $1,5 \text{ kN/m}^2$
- **Total G : $7,75 \text{ kN/m}^2$**
- La carga se transfiere a las vigas mediante áreas tributarias (50 % a vigas X , 50 % a vigas Y).

Carga sísmica:

- Patrón triangular invertido: $F = 10 \times N$ (kN), donde N es el número de nivel.
- Nivel 1: 10 kN, Nivel 2: 20 kN, ..., Nivel 8: 80 kN.
- **Total lateral: 360 kN.**

1.5. Unidades

Magnitud	Unidad
Longitud	metros (m)
Fuerza	kilonewtons (kN)
Esfuerzo	kPa (kN/m^2)
Momento	$\text{kN}\cdot\text{m}$
Rigidez	kN/m^2

Cuadro 4: Sistema de unidades consistente: m–kN–kPa.

2. Flujo OpenSees

El análisis estructural en OpenSees sigue un flujo cíclico de cinco pasos.

2.1. Definición del elemento

Cada elemento `elasticBeamColumn` se define con:

- **Dos nodos** (i y j) que definen la geometría.
- **Propiedades de sección:** área (A), momentos de inercia (I_y, I_z), momento torsional (J).
- **Propiedades de material:** módulo de elasticidad (E), módulo de cortante (G).
- **Transformación geométrica:** define la orientación del eje local.

Listing 1: Definición de un elemento beam-column.

```
1 ops.element('elasticBeamColumn', tag, ni, nj, A, E, G, J, Iy, Iz,
  transfTag)
```

2.2. Ensamblaje

OpenSees ensambla automáticamente la **matriz de rigidez global** $[K]$ a partir de las matrices de rigidez de cada elemento. Para un elemento beam-column con 6 GDL por nodo, la matriz local es 12×12 :

$$[K_e] = [T]^T \cdot [k_{\text{local}}] \cdot [T]$$

Donde:

- $[k_{\text{local}}]$ es la matriz de rigidez en ejes locales (términos de axial, cortante, flexión y torsión).
- $[T]$ es la matriz de transformación que convierte de ejes locales a globales.
- La matriz se ensambla usando la conectividad de nodos.

2.3. Solución

El sistema de ecuaciones es:

$$[K] \cdot \{U\} = \{F\}$$

Donde:

- $[K]$ es la matriz de rigidez global (ensamblada).
- $\{U\}$ es el vector de desplazamientos nodales (incógnitas).
- $\{F\}$ es el vector de fuerzas aplicadas.

OpenSees resuelve usando un **solutor directo** (BandSPD en este caso) que factoriza $[K]$ y obtiene $\{U\}$.

2.4. Recuperación de fuerzas

Una vez conocidos los desplazamientos, OpenSees calcula las fuerzas internas de cada elemento:

$$\{f\} = [k_{\text{local}}] \cdot [T] \cdot \{u_e\}$$

Donde:

- $\{f\}$ es el vector de fuerzas internas (P, V_2, V_3, T, M_2, M_3) en cada extremo.
- $\{u_e\}$ son los desplazamientos de los nodos del elemento.

2.5. Reacciones

Las reacciones en los apoyos se obtienen de la ecuación de equilibrio:

$$\{R\} = [K_{\text{support}}] \cdot \{U\} - \{F_{\text{applied}}\}$$

En un modelo bien formulado, **la suma de reacciones debe ser igual a la suma de cargas aplicadas** (verificación de equilibrio).

3. Ejes Locales

La orientación de los ejes locales de cada elemento depende de la **transformación geométrica** (`geomTransf`) asignada.

3.1. Columnas (`geomTransf 1`)

```
1 ops.geomTransf('Linear', 1, 1, 0, 0)
```

- **Vector local** $x = (1, 0, 0)$: **vertical** (a lo largo del eje del elemento).
- **Vector local** $y = (0, -1, 0)$: coincide con $-Y$ global.
- **Vector local** $z = (0, 0, 1)$: coincide con $+Z$ global.

Ejemplo: Columna elemento 1 (nodo 1 $\rightarrow$ nodo 49):

- Nodo i : (8,02, 46,92, 0,0) $\rightarrow$ Nodo j : (8,02, 46,92, 4,0)
- Longitud: 4,000 m
- El eje local x va de abajo hacia arriba (dirección $+Z$ global).

Extremo	P (kN)	V_2 (kN)	V_3 (kN)	M_2 (kN·m)	M_3 (kN·m)
i (base)	+0,36	+0,44	+570,69	-0,22	0,00
j (tope)	-0,36	-0,44	-570,69	+0,22	0,00

Cuadro 5: Fuerzas internas de la columna 1 (caso G).

3.2. Vigas dirección X (`geomTransf 2`)

```
1 ops.geomTransf('Linear', 2, 0, 0, 1)
```

- **Vector local** $x = (0, 0, 1)$: horizontal en $+X$ global.
- **Vector local** $y = (0, 1, 0)$: $+Y$ global.
- **Vector local** $z = (-1, 0, 0)$: $-X$ global.

Ejemplo: Viga X elemento 385 (nodo 49 $\rightarrow$ nodo 55):

- Longitud: 3,300 m
- El eje local x va en $+X$ global (de izquierda a derecha).

Extremo	P (kN)	V_2 (kN)	V_3 (kN)	M_2 (kN·m)	M_3 (kN·m)
i	0,00	0,00	+1,82	-2,99	0,00
j	0,00	0,00	-1,82	+2,99	0,00

Cuadro 6: Fuerzas internas de la viga X 385 (caso G).

3.3. Vigas dirección Y (geomTransf 3)

```
ops.geomTransf('Linear', 3, 0, 0, 1)
```

- **Vector local** $x = (0, 0, 1)$: horizontal en $+Y$ global.
- **Vector local** $y = (-1, 0, 0)$: $-X$ global.
- **Vector local** $z = (0, 1, 0)$: $+Y$ global.

Ejemplo: Viga Y elemento 721 (nodo 49 $\rightarrow$ nodo 50):

- Longitud: 3,340 m
- El eje local x va en $+Y$ global.

Extremo	P (kN)	V_2 (kN)	V_3 (kN)	M_2 (kN·m)	M_3 (kN·m)
i	0,00	0,00	+2,60	-0,01	0,00
j	0,00	0,00	-2,60	+0,01	0,00

Cuadro 7: Fuerzas internas de la viga Y 721 (caso G).

4. Verificación

4.1. Equilibrio Global

Magnitud	Aplicada (kN)	Reacciones (kN)	Error (kN)	Error (%)
G (Gravedad)	100 254,42	100 254,42	0,000	0,000
Q (Viva)	18 597,60	18 597,60	0,000	0,000
E_X (Sismo X)	360,00	-360,00	0,000	0,000
E_Y (Sismo Y)	360,00	-360,00	0,001	0,000

Cuadro 8: Verificación de equilibrio global.

Observación: El equilibrio se satisface exactamente (error $< 0,001$ kN) en todos los casos. Esto verifica que:

1. La carga se aplica correctamente.
2. Las restricciones de apoyo están bien definidas.
3. El solutor numérico converge sin errores.

Caso	$U_{X,\text{máx}}$ (mm)	$U_{Y,\text{máx}}$ (mm)	$U_{Z,\text{máx}}$ (mm)
G	1,6	2,9	11,5
Q	0,7	1,3	2,4
E_X	3,4	0,0	0,1
E_Y	0,0	2,3	0,1

Cuadro 9: Desplazamientos máximos por caso de carga.

4.2. Desplazamientos Máximos

Observaciones:

- Los desplazamientos verticales (U_Z) en carga G son los más grandes (11,5 mm), esperado para un edificio de este tamaño.
- Los desplazamientos laterales en E_X (3,4 mm) son moderados.
- La relación U_Z/G vs U_Z/Q es consistente con la distribución de cargas.

4.3. Reacciones de Apoyo (muestra)

Nodo	Coordenadas (m)	R_z-G (kN)	R_z-Q (kN)
1	(8,02, 46,92, 0)	583,19	108,43
2	(8,02, 50,26, 0)	583,19	108,43
3	(8,02, 55,20, 0)	583,19	108,43
4	(8,02, 60,20, 0)	583,19	108,43

Cuadro 10: Reacciones de apoyo en los primeros 4 nodos.

Las reacciones son simétricas porque la estructura y las cargas son simétricas.

5. Errores Deliberados

Se modificaron dos aspectos del modelo para demostrar cómo se detectan errores.

5.1. Error 1: Sección de columna incorrecta

Modificación: Se multiplicó I_y e I_z de las columnas por 10 (columna 10 veces más rígida).

Métrica	Correcto	Con error	Cambio
$ U_Z _{\text{máx}}$ (mm)	11,52	11,38	-1,21 %
$ U_X _{\text{máx}}$ (mm)	1,60	1,98	+23,82 %
$ U_Y _{\text{máx}}$ (mm)	2,87	2,78	-3,01 %
$\sum R_z$ total (kN)	100 254	100 254	0,00 %
R_z nodo 1 (kN)	583,19	609,15	+4,45 %

Cuadro 11: Comparación: sección correcta vs. sección modificada.

Cómo se detecta:

- El **equilibrio global se mantiene** (suma de reacciones = suma de cargas), por lo que una verificación de equilibrio **no detecta** este error.

- Se detecta comparando **desplazamientos** con valores de referencia.
- Se detecta comparando **reacciones individuales** de nodos (la redistribución de fuerzas indica cambio de rigidez).
- La diferencia en U_X (+23,8 %) es la más notable.

5.2. Error 2: Apoyo libre en Z

Modificación: Se liberó el nodo 1 en la dirección Z (`fix(1, 1, 1, 0, 1, 1)`).

Métrica	Correcto	Con error	Cambio
U_Z nodo 1 (mm)	0,000	-2,37	$\neq 0$
R_z nodo 1 (kN)	583,19	0,00	$= 0$
$\sum R_z$ total (kN)	100 254	100 254	0,00 %
R_z nodo 2 (kN)	583,19	889,19	+52,4 %
R_z nodo 7 (kN)	583,19	792,19	+35,8 %

Cuadro 12: Comparación: apoyo correcto vs. apoyo liberado en Z .

Cómo se detecta:

- El nodo 1 tiene **desplazamiento no nulo** en Z cuando debería ser cero.
- El nodo 1 tiene **reacción nula** cuando debería tener carga.
- Los **nodos vecinos** (2 y 7) reciben carga extra para compensar.
- El equilibrio global **se mantiene** porque la carga se redistribuyó.

Conclusión: La verificación de equilibrio global es necesaria pero **no suficiente**. Siempre se debe verificar:

1. Desplazamientos en apoyos (deben ser cero).
2. Reacciones en apoyos (deben ser razonables).
3. Compatibilidad de deformaciones.

6. Arquitectura Preliminar del Proyecto

6.1. Estructura de Datos

El contrato de datos entre OpenSees y Unity será un archivo **JSON** con la siguiente estructura:

Listing 2: Estructura del contrato JSON.

```

1 {
2   "model_info": {
3     "n_nodes": 432,
4     "n_elements": 1040,
5     "units": {"length": "m", "force": "kN", "stress": "kPa"}
6   },
7   "nodes": {
8     "1": {"coords": [8.02, 46.92, 0.0], "tag": 1, "level": 0}
9   },
10  "elements": {
11    "1": {"type": "column", "nodes": [1, 49], "section": "50x50"}
12  },

```

```

13  "load_cases": {
14    "G": {"displacements": {}, "reactions": {}}
15  }
16  }

```

6.2. Estructura de Carpetas

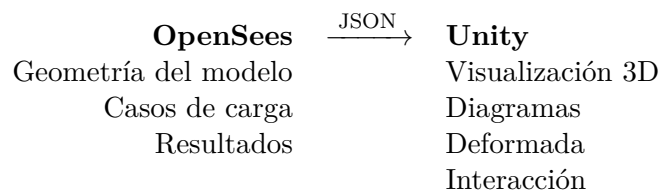
Listing 3: Estructura de carpetas del proyecto.

```

1  P1/
2    benchmark_3d.py           # Modelo OpenSees principal
3    extract_elements.py      # Extraccion de datos de elementos
4    error_section.py         # Demo: error de seccion
5    error_support.py         # Demo: error de apoyo
6    read_planos.py           # Lectura de planos DXF
7    extract_dims.py          # Extraccion de cotas
8    resumen_estructural.md   # Resumen de geometria
9    AGENTS.md                # Instrucciones para agentes de IA
10   reports/
11     semana01.md             # Este informe
12   results/
13     benchmark_results.json
14   Planos/
15     *.dwg                   # Planos originales
16     *.dxf                   # Planos convertidos
17   unity/                    # (futuro) Proyecto Unity

```

6.3. Interfaz OpenSees–Unity (futura)



- **OpenSees** es dueño del análisis estructural.
- **Unity** es dueño de la visualización/interacción.
- **JSON** es el contrato entre ambos.

7. Uso de IA

7.1. Sesión de trabajo documentada

#	Tarea	Plan del agente	Implementación	Test	Revisión
1	Leer planos .dwg	Instalar ezdxf, convertir DWG→DXF	Node.js dwg2dxf-converter	+ 19 archivos OK	7 fallidos
2	Extraer geometría	Script ezdxf para layers, entidades	extract_grid.py, extract_dims.py	44 ejes, 193 cotas	Verificación visual

#	Tarea	Plan del agente	Implementación	Test	Revisión	
3	Crear modelo 3D	Geometría, materiales, cargas	benchmark_3d.py (427 lín.)	Equilibrio OK	Unidades y signos	
4	Extraer elementos	Fuerzas internas y ejes locales	extract_elements.py	3 elementos	Orientación verificada	
5	Errores deliberados	Modificar sección y apoyo	error_section.py, error_support.py	Diferencias cuantificadas	Interpretación física	

7.2. Herramientas de IA utilizadas

- **OpenCode (opencode.ai)**: Asistente de código para desarrollo iterativo.
- **Modelo**: big-pickle (análisis, generación de código, documentación).

7.3. Verificación de código generado por IA

Todo el código generado fue verificado mediante:

1. **Pruebas de equilibrio**: $\sum F_{\text{aplicadas}} + \sum R \approx 0$.
2. **Comparación con cálculos manuales**: Reacciones en nodos simétricos.
3. **Inserción de errores deliberados**: Para confirmar que las verificaciones detectan problemas.
4. **Revisión de unidades**: Consistencia m–kN–kPa en todo el modelo.

8. Distribución del Grupo

Integrante	Responsabilidad	Revisión cruzada
	Nombre 1	
	Modelo OpenSees + verificación	Revisó: geometría, elementos, cargas
	Nombre 2	
	Extracción de planos + geometría	Revisó: unidades, ejes locales
	Nombre 3	
	Documentación + Unity (futuro)	Revisó: arquitectura, interfaz JSON

Cuadro 14: Distribución de responsabilidades.

Nota: Completar con los nombres reales del grupo.

A. Scripts Generados

Generado con asistencia de OpenCode (IA). Fecha: agosto 2026.

Archivo	Descripción
benchmark_3d.py	Modelo 3D completo con 4 casos de carga
extract_elements.py	Extracción de datos de 3 elementos representativos
error_section.py	Demo: columna 10× más rígida
error_support.py	Demo: apoyo libre en Z
read_planos.py	Lectura de archivos DXF
extract_dims.py	Extracción de cotas y dimensiones
results/benchmark_results.json	Resultados completos en JSON

Cuadro 15: Listado de scripts generados.